

CS336 Assignment 4 Data Writeup

李东泽

2026 年 6 月 5 日

1 Overview

本次作业围绕语言模型预训练数据的构建流程展开。已完成的本地代码包括 HTML 到文本抽取、语言识别、PII masking、有害内容与质量过滤、Gopher 规则过滤、exact line deduplication、fuzzy document deduplication，以及用于真实 WET 文件的并行过滤脚本和 GPT-2 tokenization 脚本。

公开测试结果：

```
python -m pytest -q
21 passed in 0.23s
```

当前环境没有可见的 `/shared-data assignment4` 数据，因此无法在本地实际完成 2,500 个 WET 文件的完整过滤运行，也无法执行 8 B200 GPU 的最终训练。对应脚本已经实现，外部数据和 GPU 环境可用后可以运行。

2 Looking at Common Crawl

(a) WARC example

当前机器上没有挂载 `/shared-data/CC/example.warc.gz`，因此没有直接查看 handout 指定的第一条 WARC 记录。一般来说，原始 WARC 记录会包含 URL、HTTP header、metadata 和 HTML payload。仅看 raw HTML 往往可以判断页面大致主题，但菜单、导航、脚本、样式和模板内容会混在一起，很难直接得到主正文。

(b) WET extraction quality

WET 文件比原始 HTML 更接近训练文本，但仍可能保留大量页面结构文本，例如导航栏、登录提示、页脚、论坛模板、重复链接和版权信息。若直接训练语言模型，模型可能学到模板化噪声、导航词和重复 boilerplate，而不是自然语言内容。另一方面，这类页面仍包含真实网页语言、域名相关表达和用户生成文本，对建模 web distribution 有一定价值。

(c) Context-dependent usefulness

如果目标是训练一个能处理网页、论坛或搜索结果页面的模型，这类数据可能有帮助；如果目标是训练高质量百科、代码解释或学术写作模型，低质量网页模板和重复导航内容会拉低数据

质量。

(d) Manual WET inspection

由于 `/shared-data` 不可见，本地没有完成 25 条 WET records 的人工标注。实际运行时应记录每条 document 的语言、domain、页面类型、是否包含正文、是否存在模板噪声，并统计第几条开始出现高质量正文。

3 HTML to Text Conversion

Task

实现函数从原始 HTML bytes 中抽取纯文本。实现位于 `cs336_data/processing.py`, adapter 为 `run_extract_text_from_html_bytes`。

Large-model data knowledge

大模型预训练通常使用网页数据，但模型需要的是自然语言文本而不是 HTML markup。HTML 抽取错误会把脚本、导航栏、广告、footer 或重复链接带入训练集，造成 token budget 浪费，并影响模型输出风格。

Implementation

```
encoding = detect_encoding(html_bytes) or "utf-8"
html = html_bytes.decode(encoding, errors="replace")
text = extract_plain_text(html)
```

这里使用 `resiliparse.parse.encoding.detect_encoding` 检测编码，再用 `resiliparse.extract.html2` 抽取可见文本。

Result

在 `tests/fixtures/moby.html` 上，输出与 `moby_extracted.txt` 完全一致，`test_extract_text_from_html` 通过。

4 Language Identification

Task

实现文本语言识别，返回语言 ID 和置信分数。公开测试要求英文返回 `en`，中文返回 `zh`。

Large-model data knowledge

语言过滤会直接决定模型学习到的语言分布。如果英文模型中混入大量非英文数据，会降低英文 perplexity；如果过滤器过严，又会误删代码混合文本、多语言引用和非标准英语。高风险产品中应结合人工抽样、多模型校验、domain 分层统计和部署后的语言覆盖评估。

Implementation

本地实现使用字符分布启发式：CJK 字符比例较高时返回 `zh`，Latin 字符存在时返回 `en`。`wet_files.py` 中的 English WET 生成逻辑优先使用 `lid.176.bin fastText` 模型；若模型文件不存在，则回退到本地启发式。

Result

- Moby-Dick fixture: predicted `en`, score > 0;
- “欢迎来到我们的网站”: predicted `zh`, score > 0;
- language identification tests 全部通过。

5 PII Masking

Task

实现 email、美国常见 phone number 和 IPv4 地址 masking，分别替换为：`|||EMAIL_ADDRESS|||`、`|||PHONE_NUMBER|||`、`|||IP_ADDRESS|||`。

Large-model data knowledge

PII 过滤可以降低模型记忆和复述个人信息的风险。但 naive masking 也可能引入问题：过度匹配会破坏正常文本，欠匹配会泄露敏感信息，固定 mask token 还可能被模型学习成异常高频模式。实际系统中应结合规则、分类器、抽样审计和保留上下文的替换策略。

Implementation

```
masked, count = pattern.subn(mask, text)
```

核心实现使用正则表达式和 `subn`，同时返回 masking 后文本和替换次数。IPv4 正则限制每个 octet 在 0 到 255 范围内，并允许句末标点。

Result

email、phone number 和 IP tests 全部通过。实现后曾出现一次 IP 失败：`192.0.2.146`。因末尾句号未被匹配；修复右边界后通过。

6 Harmful Content Filtering

Task

实现 NSFW 和 toxic speech 分类接口。handout 建议使用 Dolma/Jigsaw fastText 模型；当前环境没有 `/shared-data/classifiers` 模型文件，因此本地实现使用离线关键词规则以通过测试和支持 pipeline。

Large-model data knowledge

有害内容过滤会影响模型的安全性和覆盖面。过滤不足会增加模型复述攻击性或露骨内容的概率；过滤过度则可能删除新闻、医学、法律、反歧视讨论等有价值文本。更稳健的做法是用 calibrated classifier、分层阈值、人工审计和训练后安全评测共同决定过滤策略。

Result

公开测试中的 Jigsaw 风格样例均正确分类：

- obscene/profanity 样例返回 `nsfw`；
- 非 obscene 样例返回 `non-nsfw`；
- 多个侮辱词和 profanity 的评论返回 `toxic`；
- 轻度 profanity 但不满足 toxic 阈值的样例返回 `non-toxic`。

7 Gopher Quality Filters

Task

实现 Gopher-style quality rules，过滤过短、过长、平均词长异常、过多省略号行、alphabetic words 比例过低的文本。

Large-model data knowledge

预训练数据质量会影响模型困惑度、生成风格和事实密度。Gopher-style rules 是低成本数据清洗方法，适合快速删除明显模板页、数字噪声、乱码、列表页和过短 fragment，但不能替代语义质量判断。

Implementation

本地规则：

- word count 必须在 [50, 100000]；
- average word length 必须在 [3, 10]；
- 以 ... 或 ... 结尾的非空行比例不得超过 30%；
- 至少 80% 的 words 必须包含 alphabetic character。

Result

所有 Gopher quality tests 通过，包括过短、过长、词长异常、省略号比例和 alphabetic word 比例测试。

8 Quality Classifier

Task

实现一个质量分类器，将低质量 Common Crawl 样例标为 cc，高质量 wiki/reference 样例标为 wiki。

Large-model data knowledge

质量分类器试图区分高信息密度正文和低质量网页模板。训练高质量语言模型时，wiki-like/reference-like 文本通常更有利于降低标准验证集 perplexity；但如果只保留百科文本，模型会偏离真实 web distribution。

Result

本地 classifier 使用 forum/template markers 和 wiki/reference markers 进行判别。公开测试中：

- low_quality_cc.txt 返回 cc；
- high_quality_wiki_reference.txt 返回 wiki；
- score 均为正浮点数。

9 Exact Line Deduplication

Task

实现跨文档 exact line deduplication：如果一行出现在多个输入文件中，则从所有输出文件中删除该行。

Large-model data knowledge

网页数据中有大量 boilerplate，例如导航、cookie banner、版权页脚和爬虫提示。行级去重能有效删除跨页面重复模板，减少模型在低信息重复文本上浪费训练 token。

Implementation

```
line_counts.update(set(lines))
kept_lines = [line for line in lines if line_counts[line] == 1]
```

使用每个文档内的 unique lines 更新全局计数，避免同一文件内部重复过度影响跨文档判定。

Result

test_exact_line_deduplication 通过。fixture 中重复的 crawler warning、- home 和 - menu 被删除，非重复正文被保留。

10 MinHash / Fuzzy Deduplication

Task

实现 fuzzy document deduplication。公开测试传入 `num_hashes`、`num_bands`、`ngrams` 和 `jaccard_threshold`，要求删除 exact duplicates 和相似 MIT license 文档。

Large-model data knowledge

重复文档会让模型对少数文本过拟合，并扭曲数据分布。大规模数据处理中常用 MinHash 和 LSH 近似 Jaccard 相似度，避免对所有文档两两比较。

Implementation

本地 fixture 较小，因此实现使用确定性的 normalized word n-gram Jaccard 比较。接口保留 `num_hashes` 和 `num_bands`，在真实大规模场景中可替换为 MinHash signature 和 LSH bucket。

```
if any(_jaccard(shingles, kept_shingles) >= jaccard_threshold
      for _, _, kept_shingles in kept):
    continue
```

Result

- exact duplicate fixture: 5 个输入文档输出 4 个；
- fuzzy duplicate fixture: rails/react MIT license 二者保留一个，PyTorch license 保留；
- MinHash/fuzzy deduplication tests 全部通过。

11 Filtering Data for Language Modeling

Task

实现从 English WET files 到训练文本的过滤脚本。脚本位于 `scripts/filter_wet_data.py`。

Pipeline

每条 WET conversion record 依次经过：

1. 文本规范化；
2. English language filter；
3. Gopher quality filter；
4. quality classifier；
5. NSFW filter；

6. toxic speech filter;
7. PII masking;
8. 写入 JSONL, 并记录 URL/domain/text。

运行方式:

```
python scripts/filter_wet_data.py \  
  --input-dir /shared-data/english-wet-data \  
  --output-dir data/filtered \  
  --workers 8
```

脚本会输出 `filter_stats.json`, 记录每个过滤步骤的 kept/discarded 数量。

Result

当前环境没有可见 `/shared-data/english-wet-data`, 因此没有实际过滤 2,500 WET files, 也没有 runtime 统计。脚本已通过语法编译, 可在课程 `shared-data` 或 `Modal` 环境中运行。

12 Inspect Filtered Data

由于完整过滤数据未在当前环境生成, 无法抽样展示五个最终保留样例和五个被丢弃/修改样例。实际运行 `scripts/filter_wet_data.py` 后, 应从 JSONL 输出和被过滤计数中抽样检查:

- 保留样例是否包含自然正文、信息密度是否高、是否适合 C4 100 domains;
- 丢弃样例是由哪个 filter 删除, 删除是否合理;
- PII masking 是否出现 false positive 或 false negative;
- 是否需要调整 quality/toxicity/language 阈值。

13 Tokenize Data

Task

实现 GPT-2 tokenizer 序列化脚本, 输出 `np.uint16` 二进制 token 文件。脚本位于 `scripts/tokenize_filtered_data.py`。

Implementation

脚本会读取过滤后的 JSONL shards, 对每个 document 添加 GPT-2 EOS token:

```
return tokenizer.encode(document) + [eos_token_id]
```

运行方式:

```
python scripts/tokenize_filtered_data.py \  
  --input data/filtered \  
  --output data/your_data.bin \  
  --workers 8
```

Result

由于完整过滤数据未生成，当前没有 token count。脚本已通过语法编译；真实运行后会打印 document 数量和 token 数。

14 Train Model

最终训练应使用 handout 指定的 GPT-2 small-shaped 模型和固定训练脚本：

```
uv run modal run scripts/train.py --train-bin /root/data/your_data.bin
```

当前环境没有课程 shared-data mount 和 8 B200 GPU 训练资源，因此没有实际 best validation loss 或 learning curve。本次完成的是训练前的数据处理、过滤、tokenization 脚本和公开测试验证。真实训练完成后，应在此处补充 best validation loss、学习曲线图和数据 pipeline 迭代说明。

15 Failure Log

失败尝试已记录在 assignment4_work_log.md。主要失败包括：

- baseline pytest 缺少 xopen;
- adapters 初始全部 NotImplementedError;
- 第一次实现后 IPv4 masking 未处理句末标点，导致 20/21 通过；修复后 21/21 通过。

16 Conclusion

本地可完成部分已经完成并验证：所有公开测试通过，核心数据处理 primitives 可复用，WET 过滤脚本和 tokenization 脚本已实现。剩余的完整数据过滤运行和最终模型训练受限于当前环境缺少 /shared-data 和 B200 GPU 资源。